

Appendix for “The Wolf Pack: Response-Aware Multi-Agent Attacks for Eliciting Malicious Code”

Anonymous Authors

Table 1: Overview of existing LLM Security Approaches

(➡: Attack; : Defense; □: White-box; ■: Black-box; ☆: Discovery; ∅: Removal; : Benchmark).

Approach	Category	Acc. Model	Primary Mechanism	Key Objective
GCG [21]	➡	□	Gradient-based search for discrete adversarial suffixes.	Find transferable suffixes that maximize affirmative response probability.
PAIR [1]	➡	■	Uses an attacker LLM to iteratively refine semantic prompts.	Efficiently discover human-interpretable semantic jailbreaks in < 20 queries.
AutoDAN [10]	➡	■	Automated generation of stealthy prompt sequences.	Create jailbreaks that are transferable across different architectures.
DeepInception [9]	➡	■	“Hypnotizes” the model via virtual, nested scenes and roleplay.	Bypass safeguards by embedding harmful requests within complex fictional narratives.
Semantic Mirror (SMJ) [8]	➡	■	Genetic algorithms to optimize jailbreak prompt generation.	Concurrently optimize semantic similarity and attack success to resist metric-based filters.
Homotopy Obfuscation [6]	➡	■	Homotopy-inspired prompt obfuscation framework.	Manipulate latent behaviors to expose vulnerabilities and test resilience.
Multi-Agent Gen. [19]	➡	■	Collaborative framework of multiple autonomous agents.	Automatically generate jailbreak prompts covering various scenarios.
Prompt, Divide & Conquer [15]	➡	■	Distributed prompt processing and segmentation.	Bypass filters by dividing malicious requests and aggregating responses via an LLM jury.
Content Concretization [16]	➡	■	Iterative refinement from lower-tier to higher-tier models.	Progressively transform abstract malicious requests into production-grade executable code.
CodeJailbreaker [11]	➡	■	Encodes malicious intent in covert channels (e.g., commit messages).	Bypass instruction-following safeguards that look for explicitly articulated malicious intent.
DIJA [17]	➡	■	Interleaved Mask-Text Prompts targeting Diffusion LLMs.	Exploit bidirectional modeling and parallel decoding to force harmful completions in dLLMs.
SPELL [4]	➡	■	Sentence pairing exploration with time-division selection.	Adaptively combine sentences from prior knowledge to bypass static, template-based filters.
Red Teaming [13]	➡ (☆)	■	Uses attacker LLMs to automatically generate test cases against target LLMs.	Rapidly identify edge cases and failure modes that testers might overlook.
TwinBreak [5]	➡ (∅)	□	Iterative parameter pruning based on “twin prompt” analysis.	Permanently remove safety alignments while preserving general model utility.
MOCHA [14]	➡ /	■	Code Decomposition Attack over multiple conversational turns.	Elicit harmful code by breaking tasks into seemingly benign sub-tasks.
MalOptBench [3]	➡ /	■	Frames malicious intent as technical intelligent optimization algorithm requests.	Exploit LLM attention mechanisms that prioritize mathematical constraints over safety tokens.
MalwareBench [7]		■	Evaluates robustness using a dataset of malicious code generation prompts and combined attacks.	Systematically measure LLM code security capabilities against simple and composite jailbreak methods.
SafeDecoding [18]		Inference-time	Amplifies probabilities of safety disclaimers during decoding.	Mitigate jailbreak attacks dynamically without requiring model retraining.
Layer-AdvPatcher [12]		□	Localized unlearning using self-augmented adversarial data.	Patch “toxic layers” exhibiting high preferences for affirmative responses.
COCA [20]		Training-phase	Refactors training data with explicit reasoning processes.	Concentrate and erase non-linear harmful concepts effectively in the representation space.
D-STT [2]		Inference-time	Explicitly decodes identified “Safety Trigger Tokens” as prefixes.	Force the model onto a safety-aligned path with minimal intervention.

We search on Scopus¹ to look for relevant studies with the following query: TITLE-ABS-KEY("jailbreak" AND ("LLMs" OR "Large Language Models") AND "Code Generation") AND (LIMIT-TO (SUBJAREA, "COMP")), and the resulting papers are summarized in Table 1.

References

- [1] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," in *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2025, pp. 23–42.
- [2] H. Gu, H. Wang, Y. Mei, M. Zhang, and Y. Jin, "One Trigger Token Is Enough: A Defense Strategy for Balancing Safety and Usability in Large Language Models," Jan. 2026, arXiv:2505.07167 [cs.CR] version: 3. [Online]. Available: <http://arxiv.org/abs/2505.07167>
- [3] —, "Overlooked Safety Vulnerability in LLMs: Malicious Intelligent Optimization Algorithm Request and its Jailbreak," Jan. 2026, arXiv:2601.00213 [cs.CR] version: 1. [Online]. Available: <http://arxiv.org/abs/2601.00213>
- [4] Y. Huang, X. Jia, W. Guo, Y. Sun, Y. Huang, C. Wang, and Y. Liu, "Casting a SPELL: Sentence Pairing Exploration for LLM Limitation-breaking," Dec. 2025, arXiv:2512.21236 [cs.CR] version: 1. [Online]. Available: <http://arxiv.org/abs/2512.21236>
- [5] T. Krauß, H. Dashtbani, and A. Dmitrienko, "TwinBreak: Jailbreaking LLM Security Alignments based on Twin Prompts," Jun. 2025, arXiv:2506.07596 [cs.LG]. [Online]. Available: <http://arxiv.org/abs/2506.07596>
- [6] L. E. Lazo Vera, H. Jelodar, and R. Razavi-Far, "LLM Security and Safety: Insights from Homotopy-Inspired Prompt Obfuscation," *AI*, vol. 7, no. 3, p. 83, Mar. 2026. [Online]. Available: <https://www.mdpi.com/2673-2688/7/3/83>
- [7] H. Li, H. Gao, Z. Zhao, Z. Lin, J. Gao, and X. Li, "Llms caught in the crossfire: Malware requests and jailbreak challenges," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Association for Computational Linguistics, 2025, pp. 27 833–27 848. [Online]. Available: <https://doi.org/10.18653/v1/2025.acl-long.1350>
- [8] X. Li, S. Liang, J. Zhang, H. Fang, A. Liu, and E.-C. Chang, "Semantic Mirror Jailbreak: Genetic Algorithm Based Jailbreak Prompts Against Open-source LLMs," Feb. 2024, arXiv:2402.14872 [cs.CL] version: 2. [Online]. Available: <http://arxiv.org/abs/2402.14872>
- [9] X. Li, Z. Zhou, J. Zhu, J. Yao, T. Liu, and B. Han, "Deepinception: Hypnotize large language model to be jailbreaker," 2024. [Online]. Available: <https://arxiv.org/abs/2311.03191>
- [10] X. Liu, N. Xu, M. Chen, and C. Xiao, "Autodan: Generating stealthy jailbreak prompts on aligned large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2310.04451>
- [11] S. Ouyang, Y. Qin, B. Lin, L. Chen, X. Mao, and S. Wang, "Smoke and mirrors: Jailbreaking llm-based code generation via implicit malicious prompts," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering, ICSE 2026, Rio de Janeiro, Brasil, April 12-18, 2026*, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2503.17953>
- [12] Y. Ouyang, H. Gu, S. Lin, W. Hua, J. Peng, B. Kailkhura, M. Gao, T. Chen, and K. Zhou, "Layer-Level Self-Exposure and Patch: Affirmative Token Mitigation for Jailbreak Attack Defense," Feb. 2025, arXiv:2501.02629 [cs.CR] version: 2. [Online]. Available: <http://arxiv.org/abs/2501.02629>
- [13] E. Perez, S. Huang, F. Song, T. Cai, R. Ring, J. Aslanides, A. Glaese, N. McAleese, and G. Irving, "Red teaming language models with language models," 2022. [Online]. Available: <https://arxiv.org/abs/2202.03286>

¹<https://www.scopus.com/>

- [14] M. Wahed, X. Zhou, K. A. Nguyen, T. Yu, N. Diwan, G. Wang, D. Hakkani-Tür, and I. Lourentzou, “MOCHA: Are Code Language Models Robust Against Multi-Turn Malicious Coding Prompts?” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 22 922–22 948. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.1249/>
- [15] J. Wahréus, A. Hussain, and P. Papadimitratos, “Prompt, Divide, and Conquer: Bypassing Large Language Model Safety Filters via Segmented and Distributed Prompt Processing,” Mar. 2025, arXiv:2503.21598 [cs.CR] version: 1. [Online]. Available: <http://arxiv.org/abs/2503.21598>
- [16] —, “Jailbreaking Large Language Models Through Content Concretization,” in *Game Theory and AI for Security*, J. S. Baras, S. Papavassiliou, E. E. Tsiropoulou, and M. O. Sayin, Eds. Cham: Springer Nature Switzerland, 2026, vol. 16223, pp. 395–414, series Title: Lecture Notes in Computer Science. [Online]. Available: https://link.springer.com/10.1007/978-3-032-08064-6_20
- [17] Z. Wen, J. Qu, Z. Chen, X. Lu, D. Liu, Z. Liu, R. Wu, Y. Yang, X. Jin, H. Xu, X. Liu, W. Li, C. Lu, J. Shao, C. He, and L. Zhang, “The Devil behind the mask: An emergent safety vulnerability of Diffusion LLMs,” Feb. 2026, arXiv:2507.11097 [cs.CL] version: 2. [Online]. Available: <http://arxiv.org/abs/2507.11097>
- [18] Z. Xu, F. Jiang, L. Niu, J. Jia, B. Y. Lin, and R. Poovendran, “SafeDecoding: Defending against Jailbreak Attacks via Safety-Aware Decoding,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 5587–5605. [Online]. Available: <https://aclanthology.org/2024.acl-long.303>
- [19] A. Yang, B. Wang, A. Liu, and H. Li, “Automatically Generated Multi-Agent Framework for Jailbreaking Large Language Models,” in *International Conference on Artificial Intelligence and Industrial Technology Applications (AIITA)*, 2025.
- [20] H. Yang, Y. Chen, Z. Qin, T. Liu, C. Xiao, K. Zhang, and B. Han, “Does Representation Intervention Really Identify Desired Concepts and Elicit Alignment?” May 2025, arXiv:2505.18672 [cs.LG] version: 1. [Online]. Available: <http://arxiv.org/abs/2505.18672>
- [21] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.15043>